

docdesigner: Alpha Testing

Instructions for participants — version 0.9.1

Joseph N. Cohen

Queens College, CUNY

September 03, 2026

ABSTRACT

Thank you for taking a look at docdesigner. This alpha asks two questions and only two: **does it work on your machine and your documents, and is it easy to learn?** The visual design of the styles is deliberately unfinished and is not what we are testing yet — please read “What we are not asking about” on page 3 before you start, so you don’t spend time on problems we already know about. Setup and the first task take about fifteen minutes.

1 What this is

docdesigner renders R Markdown into publication-quality PDFs. A **style** is a small file of design tokens — typefaces, colours, spacing, rules — that the engine turns into a typeset page. The promise is *same content, any style*: you point one document at a different style and it re-typesets, without your touching a word of the prose.

Twelve styles ship, in two sets. `academic` holds `minimal`, `nature`, `methods`, `demography`, `ssrn`, `sociology` and `ajs`. `public` holds `policy`, `government`, `economist`, `atlantic` and `humanities`.

This document was itself typeset by docdesigner, in the `methods` style.

2 Setup

You need R, a LaTeX installation providing `xelatex`, and `pandoc`. RStudio bundles `pandoc`; TinyTeX and MiKTeX both work for LaTeX. If you have knitted an R Markdown document to PDF before, you already have all three.

```
1 # install.packages("remotes")
2 remotes::install_github("jncohen/docdesigner")
3
4 library(docdesigner)
5 designer_check()      # reports anything missing from the toolchain
6 designer_styles()     # the twelve styles, with their sets
```

If `designer_check()` reports a problem, fix that before going further. Nearly every confusing failure downstream is a missing `xelatex` or a missing font.

Please tell us how this step went even if it went fine — how long it took, and whether anything was unclear. Installation is the step most likely to lose a new user, and it is the one we can least easily test ourselves.

3 What to try

Three tasks, in order. The second is the important one.

3.1 Render something

Make a new R Markdown document, or take any small one you have, and put this at the top:

```
1 ---
2 title: "Anything at all"
3 author: "Your Name"
4 output:
5   docdesigner::pdf:
6     style: minimal
7 ---
```

Knit it. That is the whole interface — one line names the style. If this produces a PDF, the package works on your machine.

3.2 Render a real document of your own

This is what we most need. Take a genuine paper, report or memo of yours — the messier the better, ideally one with sections, tables, figures, footnotes and a bibliography — and render it. Then change that one `style:` line and render it again, three or four times:

```
1 output:
2   docdesigner::pdf:
3     style: nature      # then sociology, humanities, ssrn, ...
```

We want to know whether **your content survives the switch**. Tables that overflow the page, figures that land in the wrong column, citations that stop resolving, headings that collide with the running head, code that runs off the edge — that is the failure class we cannot find on our own test documents, because our test documents are tidy and yours are real.

Note that `nature` and `economist` are two-column. Wide tables and long code lines are the usual casualty

there, and we would like to know how badly.

3.3 Install a style set

```
1 designer_install_set("academic")  
2 designer_sets()
```

Sets install to your R user directory and survive package updates. This checks that distribution works from a machine that is not ours.

4 What we most want to hear

Beyond outright breakage, the thing we cannot see from here is **where you got stuck**. Specifically:

- What did you have to look up, guess at, or read twice?
- What did you expect to exist and not find?
- Did anything behave differently from how you assumed it would?
- How long did the whole thing take, start to finish?

“I assumed X and it turned out to be Y” is a more useful report than most bug reports. Please send those even when nothing was broken.

5 What we are not asking about

The rendering engine is finished and tested. The **visual design** of the styles is not, and is the next phase of work. These five are known, deliberate, and already scheduled — reporting them costs you time and tells us nothing new.

1. Every style renders a plain title block. The engine accepts several title-page layouts and currently ignores all of them. No style has a real title *page* yet.

2. economist and government do not look like their models. Both need full-bleed coloured masthead bands and boxed panels, which the engine cannot yet express. They are known to be the two weakest styles.

3. policy is being redesigned from scratch.

4. Validation reports “not yet implemented” tokens. That list is accurate and expected: those settings are recognised but not yet acted on. It is a roadmap, not a fault.

5. Fonts substitute. Proprietary typefaces are replaced by bundled open ones — XITS for Times, EB Garamond for oldstyle, Source Serif 4 for various. This is intentional.

So: please do tell us a style looks broken. Please don’t tell us it looks unfinished — we know, and we will come back to you about that in a later round.

6 How to report

Email josephncohen@gmail.com, one message per problem, with `docdesigner alpha` in the subject line.

What makes a report actionable, in rough order of value:

1. **Attach the PDF**, or a screenshot of the part that went wrong. We cannot judge a layout problem from a description.
2. **Which style**, and whether other styles do the same thing. “sociology only” and “all twelve” are different bugs.
3. **A small .Rmd that reproduces it**. If your document is confidential, cut it down until it is not — that usually localises the problem anyway.
4. **The output of `designer_check()`**, and of `sessionInfo()`.
5. **What you expected instead**.

If something failed to install, send the full console output, including the error. That is usually enough on its own.